JavaScript 101 — Complete Study Guide

Based on Your Class Assignments (Weeks 1–8)

TABLE OF CONTENTS

1. What is JavaScript and Where Does It Live?
2. Variables: `const` and `let`
3. Data Types
4. Scope: Global vs. Block
5. Console.log — Your Best Friend
6. Comments
7. Functions
8. If / Else If / Else
9. The DOM — What It Is and How to Access It
10. querySelector and querySelectorAll
11. Event Listeners
12. Changing Styles with JavaScript
13. classList — Adding, Removing, Toggling CSS Classes
14. Arrow Functions
15. Event Object (`e`)
16. Event Delegation
17. String Methods
18. Template Literals
19. Arrays
20. Objects
21. Arrays of Objects
22. forEach
23. map
24. reduce
25. Creating HTML Elements with JavaScript
26. innerHTML
27. switch Statements

28. for Loops
29. Form Handling and `preventDefault`
30. Regular Expressions (Regex)
31. The Date Object
32. Objects with Methods and `this`
33. findIndex
34. Accessibility: setAttribute and aria-label

---

1. What is JavaScript and Where Does It Live?

HTML, CSS, and JavaScript are the three building blocks of every website:

- **HTML** creates the structure — the headings, paragraphs, images, buttons.
- **CSS** styles it — colors, fonts, layout.
- **JavaScript** makes it *do things* — respond to clicks, change what you see, talk to servers.

In all your projects, JavaScript was written in a separate `.js` file and connected to the HTML page with a `<script>` tag, usually placed just before the closing `</body>` tag:

```html
<script src="scripts.js"></script>
```

Why at the bottom? So the browser fully loads all the HTML first. If the script runs before the HTML exists, JavaScript won't find any elements to work with.

---

2. Variables: `const` and `let`

A **variable** is a named container that holds a value. Think of it like a labeled box.

```javascript
const PI = 3.14;
let radius = 3;
```

`const`

Use `const` when the value will **never change**.

```javascript
const PI = 3.14;
// PI = 5; ← This would cause an ERROR. You can't reassign a const.
```

`let`

Use `let` when the value **might change later**.

```javascript
let radius = 3;
radius = 10; // This is fine. let can be reassigned.
```

Which do I use?

Default to `const`. Only switch to `let` if you know the value needs to change. This is a professional habit — it prevents accidental changes.

---

3. Data Types

Every value in JavaScript has a **type**. Here are the ones you've used:

```javascript
const PI = 3.14;        // Number
const one = 1;          // Number
const two = '2';        // String (text, always inside quotes)
let course = "CSE131";  // String
let isReady = true;     // Boolean (true or false)
```

Strings

Text wrapped in quotes. You can use single quotes `'hello'`, double quotes `"hello"`, or backticks `` `hello` `` (backticks are special — see Template Literals, section 18).

Numbers

No quotes. `3`, `3.14`, `100` are all numbers.

Important gotcha: `1` vs `'1'`

```javascript
```

```
const one = 1;     // this is the NUMBER one
const two = '2';   // this is the STRING "2"

one + 1   // → 2  (math)
two + 1   // → "21"  (JavaScript glued them together as text!)
```

This is called **type coercion** and is one of JavaScript's famous quirks.

---

4. Scope: Global vs. Block

Scope means "where can this variable be seen/used?"

```javascript
let course = "CSE131"; // global scope — visible everywhere

if (true) {
  let student = "John";
  console.log(course);  // ✅ works — course is global
  console.log(student); // ✅ works — we're inside the same block
}

console.log(course);    // ✅ works — still global
console.log(student);   // ❌ ERROR — student only exists inside the { } block
```

Global scope

A variable declared **outside** all `{ }` curly braces is global. Any code in the file can use it.

Block scope

A variable declared with `let` or `const` **inside** `{ }` curly braces only exists within those braces. Once the `}` closes, the variable is gone.

That's why you commented out the last `console.log(student)` in your Week 1 assignment — it would have thrown a `ReferenceError`.

---

5. Console.log — Your Best Friend

`console.log()` prints a message to the browser's **Developer Tools console**. It doesn't show anything on the web page itself — it's just for you (the developer) to see what's happening.

```javascript
console.log("Hello!"); // prints: Hello!
console.log(course);   // prints the value of the course variable
console.log(`${student.first} ${student.last}: ${points}`); // prints a sentence
```

How to open the console: Right-click on any web page → Inspect → Console tab.

Use `console.log()` whenever you're confused about what a variable holds, or whether your code is even running.

---

6. Comments

Comments are notes in your code that JavaScript ignores completely. They're for humans.

```javascript
// This is a single-line comment

/*
  This is a
  multi-line comment
*/

// console.log(student); // You can also "comment out" code to disable it
```

---

7. Functions

A **function** is a reusable block of code. You define it once, and then *call* it whenever you need it.

```javascript
// DEFINE the function
function changeTheme() {
    let current = selectElem.value;
    // ... do stuff
}

// CALL the function (this actually runs it)
changeTheme();
```

Functions with parameters

A **parameter** is a variable that receives a value when the function is called.

```javascript
// 'grade' is the parameter
function convert(grade) {
  let points;
  // ... figure out points based on grade
  return points;  // sends a value back to whoever called this function
}

// 'A' is the argument (the actual value passed in)
const result = convert('A'); // result is now 4
```

Return values

`return` sends a value back out of the function. Once `return` runs, the function stops immediately.

```javascript
function convert(grade) {
  if (grade === 'A') return 4;
  if (grade === 'B') return 3;
  return 0; // default
}
```

---

8. If / Else If / Else

Conditional logic lets your code make decisions.

```javascript
if (current === 'ocean') {
  // do ocean stuff
} else if (current === 'forest') {
  // do forest stuff
} else if (current === 'desert') {
  // do desert stuff
} else {
  // none of the above matched — do default stuff
}
```

The `===` operator (strict equality)

`===` checks that two values are **exactly equal** — same value AND same type.

```javascript
'ocean' === 'ocean'   // true
'ocean' === 'Ocean'   // false (case sensitive!)
1 === '1'             // false (different types: number vs string)
1 == '1'              // true  (== is looser — avoids this)
```

Always use `===` instead of `==` unless you have a specific reason not to.

Other comparison operators

```javascript
x > 5    // greater than
x < 5    // less than
x >= 5   // greater than or equal to
x <= 5   // less than or equal to
x !== 5  // NOT equal to
```

Combining conditions

```javascript
if (x > 0 && x < 10) { }   // AND — both must be true
if (x < 0 || x > 100) { }  // OR — at least one must be true
```

---

9. The DOM — What It Is and How to Access It

DOM stands for Document Object Model. It's the browser's internal representation of your HTML page as a tree of objects that JavaScript can interact with.

When the browser reads your HTML file, it creates a JavaScript object for every element — every `<h1>`, every `<p>`, every `<button>`. JavaScript can then read, modify, create, or delete any of them.

Think of it this way:

- HTML is the blueprint.
- The DOM is the actual building the browser constructed from that blueprint.
- JavaScript is you walking through the building and rearranging furniture.

10. querySelector and querySelectorAll

These are how you "grab" elements from the page so JavaScript can work with them.

```javascript
// Grab ONE element — returns the FIRST match
const nav = document.querySelector('nav');          // by tag name
const btn = document.querySelector('.menu-btn');     // by class (use .)
const modal = document.querySelector('dialog');      // by tag
const select = document.querySelector('#theme-select'); // by ID (use #)
```

The selector syntax is identical to CSS selectors:

- `'nav'` → grab the `<nav>` element
- `'.menu-btn'` → grab element with `class="menu-btn"`
- `'#theme-select'` → grab element with `id="theme-select"`
- `'article.movie p span'` → grab `<span>` inside `<p>` inside `<article class="movie">`

querySelectorAll

Returns **all matching elements** as a NodeList (like an array).

```javascript
const allRatingElems = document.querySelectorAll('article.movie p span');
// allRatingElems now contains EVERY span inside .movie paragraphs
```

Getting things from an element

```javascript
element.value       // the current value of an input or select
element.src         // the src attribute of an image
element.alt         // the alt attribute of an image
element.tagName     // the tag name in UPPERCASE: 'IMG', 'DIV', 'BUTTON'
element.innerText   // the visible text content of an element
element.textContent // all text content (including hidden)
element.innerHTML   // the full HTML content inside the element
element.offsetHeight // forces browser to recalculate layout (used in animation tricks)
```

11. Event Listeners

An **event listener** watches for something to happen (a click, a change, a keypress) and runs a function when it does.

```javascript
// Syntax:
element.addEventListener('event-name', functionToRun);
```

Common event types you used:

Event	When it fires
`'click'`	User clicks the element
`'change'`	User changes a select/input value
`'submit'`	User submits a form
`'resize'`	The browser window is resized

Example from your projects:

```javascript
selectElem.addEventListener('change', changeTheme);
// When the select changes, run the changeTheme function

toggleButton.addEventListener('click', () => {
  nav.classList.toggle('active');
});
// When the button is clicked, run this arrow function
```

The function passed to `addEventListener` is called a **callback** — it's a function you give to something else to call later.

---

12. Changing Styles with JavaScript

You can change CSS styles directly from JavaScript using `element.style`.

```javascript
document.body.style.backgroundImage = "url('ocean.jpg')";
pageContent.style.fontFamily = "Papyrus, fantasy";
pageContent.style.color = 'rgb(255, 255, 255)';
textBackground.style.backgroundColor = 'rgb(38, 135, 157)';
logo.style.opacity = '0.5';
logo.style.width = '85px';
logo.src = 'byui-logo-white.png'; // ← changing an attribute, not a style
```

CSS property names in JavaScript

CSS uses hyphens: `background-color`, `font-family` JavaScript uses camelCase: `backgroundColor`, `fontFamily`

Whenever you see a hyphen in CSS, remove it and capitalize the next letter for JavaScript.

---

13. classList — Adding, Removing, Toggling CSS Classes

Instead of setting inline styles directly, a cleaner approach is to toggle CSS classes on and off. You define the style in your CSS file, then JavaScript just adds or removes the class name.

```javascript
element.classList.add('show');     // adds the class "show"
element.classList.remove('show');  // removes the class "show"
element.classList.toggle('show');  // if it has it, remove it; if not, add it
```

Example from your navigation projects:

```javascript
// CSS:
// nav.show { display: block; }

// JavaScript:
btn.addEventListener('click', function() {
    nav.classList.toggle('show');
    // First click: nav gets class "show" → appears
    // Second click: "show" is removed → disappears
});
```

Checking if a class exists:

```javascript
element.classList.contains('show') // returns true or false
```

---

14. Arrow Functions

Arrow functions are a shorter way to write functions. You used both regular functions and arrow functions.

```javascript
// Regular function (named)
function changeTheme() {
  // ...
}

// Regular anonymous function
selectElem.addEventListener('change', function() {
  // ...
});

// Arrow function (anonymous)
selectElem.addEventListener('change', () => {
  // ...
});

// Arrow function with a parameter
gallery.addEventListener('click', (e) => {
  // e is the event object
});
```

Arrow functions are especially popular for short callbacks. They're the same idea as regular functions — just shorter to write.

---

15. Event Object (`e`)

When an event fires, JavaScript automatically creates an **event object** that contains information about what happened. You catch it by naming a parameter (usually `e` or `event`).

```javascript
```

```
function openModal(e) {
  console.log(e);         // the full event object
  console.log(e.target);   // the specific element that was clicked/changed
  console.log(e.target.tagName); // the tag name of that element
  console.log(e.target.src);     // if it's an image, its src
  console.log(e.target.value);   // if it's an input/select, its value
}

gallery.addEventListener('click', openModal);
```

`e.target` vs `e.currentTarget`

- `e.target` — the exact element the user interacted with
- `e.currentTarget` — the element that has the event listener attached

These can be different when you use event delegation (next section).

---

16. Event Delegation

Instead of putting a click listener on every individual image, you put ONE listener on the parent, and let click events "bubble up" to it.

```javascript
// Instead of putting a listener on each image...
// Put ONE listener on the whole gallery container:
gallery.addEventListener('click', openModal);

function openModal(e) {
  // Check if what was actually clicked was an image
  if (e.target.tagName !== 'IMG') return;
  // If not an image, stop here. 'return' exits the function early.

  // If we're here, the user clicked an image
  const smallSrc = e.target.src;
  // ...
}
```

Why? Efficiency. If you have 50 images, you'd need 50 event listeners. With delegation, you need just 1.

The `return` statement on its own (with no value) is used here as an **early exit** — it stops the function from doing anything if the condition isn't met.

17. String Methods

Strings have built-in tools (methods) you can call on them.

```javascript
// .replace(old, new) — replaces part of a string
const smallSrc = 'photo-sm.jpg';
const largeSrc = smallSrc.replace('-sm', '-full');
// largeSrc is now 'photo-full.jpg'

const largeSrc2 = smallSrc.replace('.jpg', '-full.jpg');
// largeSrc2 is now 'photo-sm-full.jpg'

// .trim() — removes whitespace from start and end
const input = '   hello   ';
input.trim(); // → 'hello'

// .length — how many characters in the string
'hello'.length;   // → 5
'⭐⭐⭐'.length; // → 3

// .toFixed(2) — rounds a number to 2 decimal places, returns a string
(3.14159).toFixed(2); // → '3.14'
gpa.toFixed(2);      // → '3.33' (for example)
```

---

18. Template Literals

Template literals use **backticks** (`) instead of quotes and let you embed JavaScript expressions directly inside strings using `${ }`.

```javascript
// Old way (clunky):
li.textContent = student.first + ' ' + student.last + ' - Grade: ' + student.grade;

// New way with template literals:
li.textContent = `${student.first} ${student.last} - Grade: ${student.grade}`;
```

You can put any JavaScript expression inside `${ }`:

```javascript
`Class GPA: ${gpa.toFixed(2)}`
`${rating} out of 5 stars`
`Course Name: ${aCourse.name}`
```

Template literals also support **multi-line strings**:

```javascript
const html = `
  <h2>${movie.title}</h2>
  <p>Release Date: ${movie.date}</p>
  <p>Genre: ${movie.genre}</p>
`;
```

This is extremely useful for building HTML strings in JavaScript.

---

19. Arrays

An **array** is an ordered list of values, wrapped in `[ ]`.

```javascript
const words = ['watermelon', 'peach', 'apple', 'tomato', 'grape'];
```

- `words[0]` → `'watermelon'` (arrays start at index 0)
- `words[1]` → `'peach'`
- `words.length` → `5`

Arrays can hold anything: strings, numbers, objects, even other arrays.

---

20. Objects

An **object** is a collection of related data stored as **key: value** pairs, wrapped in `{ }`.

```javascript
```

```javascript
const student = {
  last: 'Andrus',
  first: 'Aaron',
  grade: 'A'
};
```

Access values using dot notation:

```javascript
student.first   // → 'Aaron'
student.last    // → 'Andrus'
student.grade   // → 'A'
```

Or bracket notation:

```javascript
student['first']   // → 'Aaron' (same result, useful when the key is a variable)
```

---

21. Arrays of Objects

This is one of the most important and common patterns in JavaScript. An array where every item is an object.

```javascript
const students = [
  { last: 'Andrus', first: 'Aaron', grade: 'A' },
  { last: 'Masa', first: 'Manny', grade: 'C' },
  { last: 'Tanda', first: 'Tamanda', grade: 'B' }
];
```

You used this pattern in almost every project from Week 6 onward: movies, books, sections, the character object.

```javascript
students[0]        // → { last: 'Andrus', first: 'Aaron', grade: 'A' }
students[0].first   // → 'Aaron'
students[1].grade   // → 'C'
```

---

22. forEach

`forEach` loops over every item in an array and runs a function for each one.

```javascript
// Syntax:
array.forEach(function(item) {
  // do something with item
});

// Arrow function version (more common):
array.forEach((item) => {
  // do something with item
});
```

Example from your projects:

```javascript
students.forEach(student => {
  const points = convert(student.grade);
  const li = document.createElement('li');
  li.textContent = `${student.first} ${student.last} - Grade: ${student.grade}`;
  studentList.appendChild(li);
});
```

This runs the callback once for `students[0]`, once for `students[1]`, once for `students[2]`.

---

23. map

`map` is like `forEach`, but it **creates a new array** from the results. Whatever you `return` from the callback becomes an item in the new array.

```javascript
// Convert grades to points
const pointsArray = students.map(student => convert(student.grade));
// If grades are A, C, B → pointsArray is [4, 2, 3]
```

Think of it as transforming every item in the array into something else.

```javascript
```

```
// Another example from your projects:
const html = sections.map(sectionTemplate);
// Takes each section object and turns it into an HTML string
```

---

24. reduce

`reduce` collapses an entire array down to **a single value** (like a sum, or a combined string).

```javascript
const totalPoints = pointsArray.reduce((total, points) => total + points, 0);
```

Breaking this down:

- `total` — the running accumulated value (starts at `0`, the second argument)
- `points` — the current item in the array
- Each call: `total = total + points`
- After all items: `total` is the final sum

For `[4, 2, 3]`:

- Start: `total = 0`
- Step 1: `total = 0 + 4 = 4`
- Step 2: `total = 4 + 2 = 6`
- Step 3: `total = 6 + 3 = 9`
- Result: `9`

Then: `const gpa = totalPoints / students.length;` → `9 / 3 = 3`

---

25. Creating HTML Elements with JavaScript

Instead of writing HTML manually, you can create elements with JavaScript and insert them into the page.

```javascript
```

```
// 1. Create a new element
const li = document.createElement('li');

// 2. Set its content
li.textContent = `${student.first} ${student.last}`;

// 3. Optionally create a child element
const starSpan = document.createElement('span');
starSpan.textContent = ' ⭐ ⭐ ⭐ ';

// 4. Append the child to the parent
li.appendChild(starSpan);

// 5. Append the whole thing to the page
studentList.appendChild(li);
```

`appendChild` inserts an element as the **last child** of another element.

This is powerful because you can create and insert elements dynamically based on data (arrays/objects), rather than hard-coding them in HTML.

---

26. innerHTML

`innerHTML` lets you set (or get) the HTML content inside an element as a string.

```javascript
element.innerHTML = `<h2>${movie.title}</h2><p>${movie.description}</p>`;
```

This is useful when you're generating a chunk of HTML at once.

Difference between textContent, innerText, and innerHTML:

```javascript
element.textContent = '<b>Hello</b>';  // displays literally: <b>Hello</b>
element.innerText = '<b>Hello</b>';    // displays literally: <b>Hello</b>
element.innerHTML = '<b>Hello</b>';    // displays: Hello (bold)
```

`innerHTML` actually parses and renders the HTML. `textContent` and `innerText` treat everything as plain text.

Replacing page content

In your form projects, you replaced the whole form with a thank-you message:

```javascript
formContainer.innerHTML = '<p>Thank you for your purchase.</p>';
```

---

27. switch Statements

A `switch` statement is like a cleaner version of a long `if / else if` chain when you're checking one variable against many possible values.

```javascript
switch (grade) {
  case 'A':
    points = 4;
    break;
  case 'B':
    points = 3;
    break;
  case 'C':
    points = 2;
    break;
  case 'D':
    points = 1;
    break;
  case 'F':
    points = 0;
    break;
  default:
    alert('not a valid grade');
    return 0;
}
```

- Each `case` is a value to compare against
- `break` stops JavaScript from "falling through" to the next case
- `default` runs if no case matched (like the `else` in an if/else)

---

28. for Loops

A `for` loop repeats code a set number of times.

```javascript
let stars = '';
for (let i = 0; i < points; i++) {
  stars += '⭐';
}
```

Breaking this down:

- `let i = 0` — start counter at 0
- `i < points` — keep going as long as `i` is less than `points`
- `i++` — after each loop, add 1 to `i` (same as `i = i + 1`)
- `stars += '⭐'` — adds a star to the string (same as `stars = stars + '⭐'`)

If `points = 3`, the loop runs 3 times → `stars = '⭐⭐⭐'`

---

29. Form Handling and `preventDefault`

When a form is submitted, by default the browser **reloads the page** and sends data to a server. In your projects, you wanted to handle everything in JavaScript instead.

```javascript
theForm.addEventListener('submit', submitHandler);

function submitHandler(event) {
  event.preventDefault(); // ← STOP the default browser behavior (page reload)

  // Now do your own validation and logic
}
```

Validating inputs

```javascript
```

```
const cardNum = document.querySelector('#creditCardNumber').value.trim();

if (cardNum === '') {
    errorMsg += 'Card number is required\n';
}
```

Showing errors

```javascript
function displayError(msg) {
    document.querySelector('.errors').textContent = msg;
}

displayError('Card number must be 16 digits');
```

Hiding/showing sections with classes

```javascript
creditCardContainer.classList.add('hide');    // CSS: .hide { display: none; }
creditCardContainer.classList.remove('hide'); // makes it visible again
```

---

30. Regular Expressions (Regex)

A **regular expression** is a pattern used to test or match text.

```javascript
// Test if cardNum is exactly 16 digits
if (!/^\d{16}$/.test(cardNum)) {
    errorMsg += 'Card number must be 16 digits\n';
}
```

Breaking down `!/^\d{16}$/.test(cardNum)`:

- `/^\d{16}$/` is the regex pattern wrapped in `/` slashes
- `^` means "start of string"
- `\d` means "any digit (0-9)"
- `{16}` means "exactly 16 times"

- `$` means "end of string"
- `.test(cardNum)` runs the test — returns `true` if the string matches
- `!` inverts it — so `!` + `true` = `false`, meaning "if the number does NOT match this pattern"

```javascript
// Another example from your project:
if (!/^15\d{14}$/.test(cardNum)) {
  // Must start with "15" then have 14 more digits
}
```

---

31. The Date Object

JavaScript has a built-in `Date` object for working with dates and times.

```javascript
const currentDate = new Date(); // creates an object for right now

currentDate.getFullYear()  // → 2026 (the 4-digit year)
currentDate.getMonth()     // → 0–11 (January is 0, December is 11)
currentDate.getDate()      // → 1–31 (day of month)
```

Checking if a card is expired:

```javascript
const expYear = 2000 + Number(document.querySelector('#year').value);
const expMonth = Number(document.querySelector('#month').value);
const currentDate = new Date();

if (
  expYear < currentDate.getFullYear() ||
  (expYear === currentDate.getFullYear() && expMonth <= currentDate.getMonth() + 1)
) {
  errorMsg += 'Card is expired\n';
}
```

Why `getMonth() + 1`? Because `getMonth()` returns 0 for January, but humans write months as 1–12. Adding 1 converts it.

`Number(...)` converts a string to a number. `document.querySelector('#year').value` always returns a string.

32. Objects with Methods and `this`

An object can contain functions. When a function is inside an object, it's called a **method**.

```javascript
const aCourse = {
  code: 'CSE121b',
  name: 'Javascript Language',
  sections: [
    { sectionNum: 1, roomNum: 'STC 353', enrolled: 26, days: 'TTh', instructor: 'Bro T'},
    // ...
  ],
  enrollStudent: function(sectionNum) {
    // 'this' refers to the object itself (aCourse)
    const sectionIndex = this.sections.findIndex(
      (section) => section.sectionNum == sectionNum
    );
    if (sectionIndex >= 0) {
      this.sections[sectionIndex].enrolled++;
      renderSections(this.sections);
    }
  }
};
```

`this`

Inside an object's method, `this` refers to the object itself. It's like saying "my own properties."

```javascript
this.sections       // same as aCourse.sections
this.sections[0].enrolled++ // adds 1 to the enrolled count
```

Calling methods:

```javascript
aCourse.enrollStudent(2);  // calls the enrollStudent method on aCourse
character.attacked();      // calls the attacked method on character
character.levelUp();
character.reset();
```

33. findIndex

`findIndex` searches an array and returns the **index** (position number) of the first item that matches a condition. Returns `-1` if nothing matched.

```javascript
const sectionIndex = this.sections.findIndex(
  (section) => section.sectionNum == sectionNum
);
// If sectionNum is 2, it searches for the section where .sectionNum equals 2
// Returns 1 (the index in the array), or -1 if not found

if (sectionIndex >= 0) {
  // found it! sectionIndex is 0, 1, or 2
  this.sections[sectionIndex].enrolled++;
}
```

Compare to `find`, which returns the actual item (not the index):

```javascript
const section = this.sections.find(s => s.sectionNum == sectionNum);
// Returns the section object itself, not its position
```

---

34. Accessibility: setAttribute and aria-label

`setAttribute` sets any HTML attribute on an element.

```javascript
starSpan.setAttribute('aria-label', `${points} out of 4 grade points`);
elem.setAttribute('aria-label', `${rating} out of 5 stars`);
```

Why aria-label?

Emoji stars (⭐⭐⭐⭐) look great visually, but a screen reader for a visually impaired user would read each emoji character's name out loud: "white medium star, white medium star, white medium star..." — not useful.

The `aria-label` attribute overrides what the screen reader says. With `aria-label="4 out of 5 stars"`, the screen reader announces "4 out of 5 stars" — much better.

How the rating was calculated:

```javascript
const rating = elem.innerText.length;
// '⭐⭐⭐⭐'.length === 4 (four star characters)
elem.setAttribute('aria-label', `${rating} out of 5 stars`);
```

QUICK REFERENCE: Patterns Used Over and Over

Grab an element and listen for a click:

```javascript
const btn = document.querySelector('#myButton');
btn.addEventListener('click', function() {
  // do something
});
```

Loop through data and build HTML:

```javascript
const items = [...]; // array of objects
items.forEach((item) => {
  const li = document.createElement('li');
  li.textContent = item.name;
  list.appendChild(li);
});
```

Toggle a class to show/hide something:

```javascript
// CSS: .show { display: block; }
element.classList.toggle('show');
```

Build HTML from an object:

```javascript
```

```
article.innerHTML = `
    <h2>${movie.title}</h2>
    <p>${movie.description}</p>
`;
```

Prevent form reload and validate:

```javascript
form.addEventListener('submit', function(e) {
    e.preventDefault();
    const value = input.value.trim();
    if (value === '') {
        errorElem.textContent = 'This field is required';
        return;
    }
    // success logic
});
```

WEEK-BY-WEEK SUMMARY

Week	Key Concepts Introduced
1	Variables (`const`, `let`), data types, scope, `console.log`, comments
2	`querySelector`, `addEventListener`, `if/else`, changing styles, `classList`
3	Arrow functions, `classList.toggle`, `window.resize`, `window.innerWidth`
4	Event delegation, `e.target`, `tagName`, string `.replace()`, `showModal()`, CSS transitions
5	`querySelectorAll`, `forEach`, `innerText`, `setAttribute`, `aria-label`
6	Arrays, objects, arrays of objects, `map`, `reduce`, `createElement`, `appendChild`, `innerHTML`, `switch`, `for` loops, template literals
7	Form handling, `preventDefault`, regex, `Date` object, `Number()`
8	Objects with methods, `this`, `findIndex`, rendering from data

Good luck on your test! You've actually covered a tremendous amount — if you understand every section in this guide, you're well prepared.